arXiv:2506.00714v1 [cs.SE] 31 May 2025

# An LLM Agent for Functional Bug Detection in Network Protocols

Mingwei Zheng[1], Chengpeng Wang[1], Xuwei Liu[1], Jinyao Guo[1], Shiwei Feng[1],and Xiangyu Zhang[1]
[1]Department of Computer Science, Purdue University, West Lafayette, USA
Email: {zheng618, wang6590, liu2598, guo846, feng292, xyzhang}@purdue.edu

***Abstract*—Functional correctness is critical for ensuring the reliability and security of network protocol implementations. Functional bugs, instances where implementations diverge from behaviors specified in RFC documents, can lead to severe consequences, including faulty routing, authentication bypasses, and service disruptions. Detecting these bugs requires deep semantic analysis across specification documents and source code, a task beyond the capabilities of traditional static analysis tools. This paper introduces RFCSCAN, an autonomous agent that leverages large language models (LLMs) to detect functional bugs by checking conformance between network protocol implementations and their RFC specifications. Inspired by the human auditing procedure, RFCSCAN comprises two key components: an indexing agent and a detection agent. The former hierarchically summarizes protocol code semantics, generating semantic indexes that enable the detection agent to narrow down the scanning scope. The latter employs demand-driven retrieval to iteratively collect additional relevant data structures and functions, eventually identifying potential inconsistencies with the RFC specifications effectively. We evaluate RFCSCAN across six real-world network protocol implementations. RFCSCAN identifies 47 functional bugs with 81.9% precision, of which 20 bugs have been confirmed or fixed by developers.**

## I. INTRODUCTION

Network protocols are essential to digital communication, establishing standardized rules that govern data exchange between devices. They specify message formats and sequencing, as well as mechanisms for routing, connection management, and resource allocation. Due to their complexity, implementations of network protocols are susceptible to functional bugs—logical errors that cause deviations from intended behaviors, which undermines the reliability and security of network infrastructures. For example, the Zerologon vulnerability (CVE-2020-1472 [1]) in Microsoft's Netlogon Remote Protocol (MS-NRPC) [2] stemmed from improper initialization of a cryptographic vector, which allows attackers to impersonate any host, including domain controllers, and gain unauthorized domain administrator access. Due to its critical severity, it was assigned a maximum CVSS score of 10.0.

To enhance the reliability and security of network protocols, it is essential to identify functional bugs in their implementations. Manually detecting such bugs is notoriously difficult, as developers have to compare the implementation against every relevant clause in lengthy RFC documents—a process that demands significant human effort and domain expertise. However, automating functional bug detection remains a challenging task. First, the semantic properties depicting protocol functionality are highly diverse and expressed informally in

**RFC 8966 3.8.2.1. Avoiding Starvation**

If a Babel node loses all feasible routes to a destination but still has unexpired **unfeasible** routes, it **must** send a **sequence number request** (seqno request) to attempt route recovery.

(a) Babel specification

```
void route_lost(struct source *src, unsigned oldmetric) {
   new_route = find_best_route(src->prefix, src->plen, 1, NULL);
   if (new_route) { consider_route(new_route); }
   else {
-    if (oldmetric < INFINITY) {
+      unfeasible = find_best_route(src->prefix, src->plen, 0, NULL);
+    if (unfeasible && !route_expired(unfeasible))
+      send_request_resend(NULL, src->prefix, src->plen, …);
+    else if (oldmetric < INFINITY)
        send_update_resend(…);  ..
    } ..
```

(b) The bug fix in FRRouting

**Fig. 1: An example functional bug in Babel and its fix**

natural language within RFC documents. In contrast to generic low-level safety properties [3], which can often be uniformly represented using logical constraints, application-specific high-level semantic properties (or *semantic properties* in this paper), such as those found in network protocols, are considerably more difficult to formalize due to their reliance on domain knowledge and context-dependent behaviors. As shown in the example in Figure 1(a), such properties in the RFC document often require reasoning about the states of multiple protocol entities and the complex interactions among them, making encoding such properties inherently intricate. Second, the diversity of semantic properties significantly hinders the development of effective detection mechanisms. Conventional bug detection techniques, such as constraint-based methods (e.g., symbolic execution [4]–[6]) and graph-based reasoning approaches (e.g., data-flow reachability analysis [7], [8]), are inadequate for automatically validating the semantic properties of network protocols. These methods cannot automatically localize the functions relevant to specific properties and verify their compliance with the specification. For instance, it is challenging to localize the function `route_lost` shown in Figure 1(b) and leverage a uniform reasoning framework to detect violations of the associated semantic property.

Although recent studies have proposed various techniques for bug detection in network protocol implementations, they remain inadequate to effectively identify functional bugs. In general, existing approaches fall into three categories. First, fuzzing techniques aim to uncover bugs by generating crash-triggering inputs [9], [10]. However, functional bugs typically

do not cause crashes or other observable anomalies, making them difficult for fuzzing-based methods to detect. Second, differential analysis can reveal behavioral inconsistencies and discover potential functional bugs by comparing multiple independent implementations of the same protocol [11]–[13]. Yet, this approach depends on the availability of high-quality alternative implementations and does not guarantee conformance to the original specification, thereby limiting its recall for functional bug detection. Third, formal verification offers strong semantic guarantees but requires rigorous formalization of protocol properties and formal reasoning frameworks [14], [15]. These requirements may not be easily achievable for real-world protocols due to the informal and diverse specifications highlighted above. These limitations suggest the need for a new paradigm of bug detection that enables semantic reasoning on both informal specifications and implementation behavior, an essential capability for effectively detecting functional bugs.

This paper presents RFCSCAN, the first LLM-based agent for detecting functional bugs in network protocol implementations. Our approach is motivated by the observation that LLMs exhibit strong capabilities in understanding both natural language specifications, such as RFC documents, and the semantics of code snippets when they are well scoped, such as those spanning just a few related functions. Notably, RFC documents are typically well-structured and articulate protocol specifications clearly in natural language. Meanwhile, protocol implementations often follow intuitive naming conventions that reflect the functionality of functions and modules. This suggests a promising opportunity: Given an informal property described in an RFC, we can leverage LLMs to align the natural language description of the property with the code structure. If the functions related to the property can be precisely identified by further code retrieval (e.g., along calling contexts), LLMs can perform comprehensive and accurate reasoning over these functions to discover potential functional inconsistencies, thereby enabling functional bug detection. Technically, RFCSCAN consists of two collaborative agents, namely an ***indexing agent*** and a ***detection agent***, which perform the following core analyses, respectively.

- *Code Semantic Indexing.* To bridge informal properties in RFC documents with relevant code scope, the indexing agent constructs hierarchical semantic indexes of the protocol implementation using LLMs, which summarizes the semantics of functions, files, and directories into concise natural language descriptions. Such semantic indexes allow RFCSCAN to narrow down the scope of functions relevant to the target property, significantly reducing token and time costs during the detection.
- *Retrieval-Guided Detection.* To collect sufficient contexts, the detection agent progressively retrieves additional relevant functions using symbolic tools driven by the LLM. If the currently available context suffices to validate or refute conformance to the semantic property, RFCSCAN proceeds to the next property. Otherwise, it augments the context by retrieving callers or callees until a conclusion can be drawn. The retrieved functions can also support self-critics in the detection, during which the detection agent reviews its own reasoning steps upon the retrieved functions to mitigate hallucinations and improve the precision of the detection.

We implement RFCSCAN as a prototype, powered by Claude 3.5 Sonnet. We apply RFCSCAN to the implementations of six widely-used network protocols, including Babel, DHCP, and IGMP, and assess its ability to detect functional bugs according to corresponding RFC documents. It is shown that RFCSCAN successfully identifies 47 zero-day functional bugs with a high precision of 81.9%. In comparison, existing advanced agents, such as Copilot powered with three advanced models: Claude 3.7 Sonnet, Claude 3.5 Sonnet, and GPT-4o, only identify 26 bugs on average, while exhibiting a substantially higher average false positive rate of 77.5%. We further conduct a comprehensive ablation study, which highlights the significant precision gains attributed to the designs of code semantic indexing and retrieval-guided detection. As of the time of submission, 20 of the detected bugs have been confirmed or fixed by the respective developers. In summary, our work makes the following contributions:

- We propose RFCSCAN, the first LLM agent that checks the compliance of network protocol implementation and RFC documents for functional bug detection.
- We introduce a multi-agent design that summarizes code functionalities as semantic indexes and leverages them for demand-driven code retrieval, facilitating functional bug detection with high precision.
- We evaluate RFCSCAN on six network protocol implementations and identify 47 functional bugs with 81.9% precision, outperforming existing LLM-based baselines in both precision and semantic coverage.
- A prototype implementation of RFCSCAN is publicly available at https://github.com/zmw12306/RFCScan.

## II. OVERVIEW

In this section, we begin with a real-world functional bug in network protocol and discuss the limitations of existing approaches (Section II-A). Then we provide the key ideas of our approach (Section II-B) with illustrative examples.

### A. Motivating Example

Figure 1 shows a real-world functional bug in the implementation of the protocol Babel, a collaborative project under the Linux Foundation with over 400 contributors, powering major open-source large-scale networking platforms like Microsoft's SONiC and Amazon's DENT. According to its RFC document shown in Figure 1 (a), when a node loses all feasible routes to a destination but still holds unexpired unfeasible routes, it MUST send a sequence number (seqno) request to trigger route recovery. An unfeasible route means a neighbor recently advertised a path that is not currently usable, but might become valid again. Sending a seqno request prompts that neighbor to re-announce the route, potentially restoring connectivity. However, the original implementation fails to check for unfeasible routes, which can leave the router in a starved state

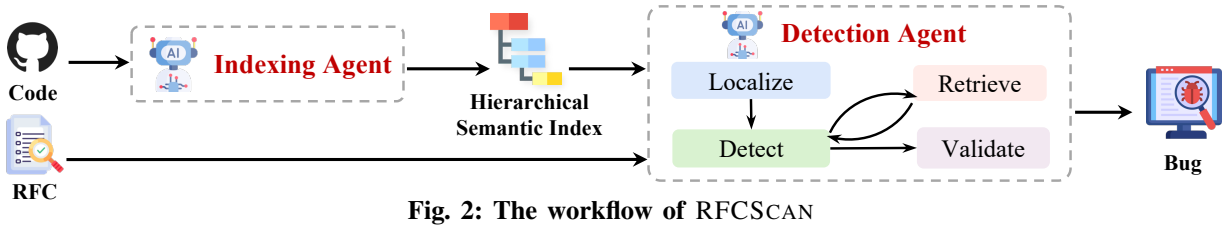



**Fig. 2: The workflow of RFCSCAN**

with no way to recover, causing persistent connectivity loss in dynamic networks. As shown in Figure 1 (b), the fix adds an explicit check for unexpired unfeasible routes and issues a seqno request when such a route is found, ensuring RFC compliance and preventing persistent routing failure. This bug has been confirmed and fixed by the developers.

However, detecting functional bugs is extremely challenging. Traditional bug detection methods, such as fuzzing [9], [16], [17] and static analysis [18], [19], primarily focus on identifying low-level issues like memory corruption. Consequently, these approaches are ineffective against non-crashing protocol non-compliance, such as the bug illustrated in Figure 1. Model checking techniques also encounter significant difficulties in detecting this bug, primarily due to their dependence on explicitly specified and formally defined properties. For instance, tools such as CMC [15] and UPPAAL [20] efficiently verify high-level behaviors like state transitions and message sequencing, while struggling to capture diverse and nuanced semantic properties, exemplified in Figure 1(a). Moreover, even when semantic properties can be explicitly specified, model checking requires extensive manual abstraction efforts, which are often labor-intensive and susceptible to errors, constraining their practical applicability. Lastly, recent studies such as ParDiff [11] and ParCleanse [21] target non-crashing parsing bugs by inferring valid packet formats from code or RFCs. While effective for identifying parser-related issues, these approaches fall short in detecting functional bugs that involve more complex semantic properties in components beyond network packet parsers.

### *B.* RFCSCAN *in a Nutshell*

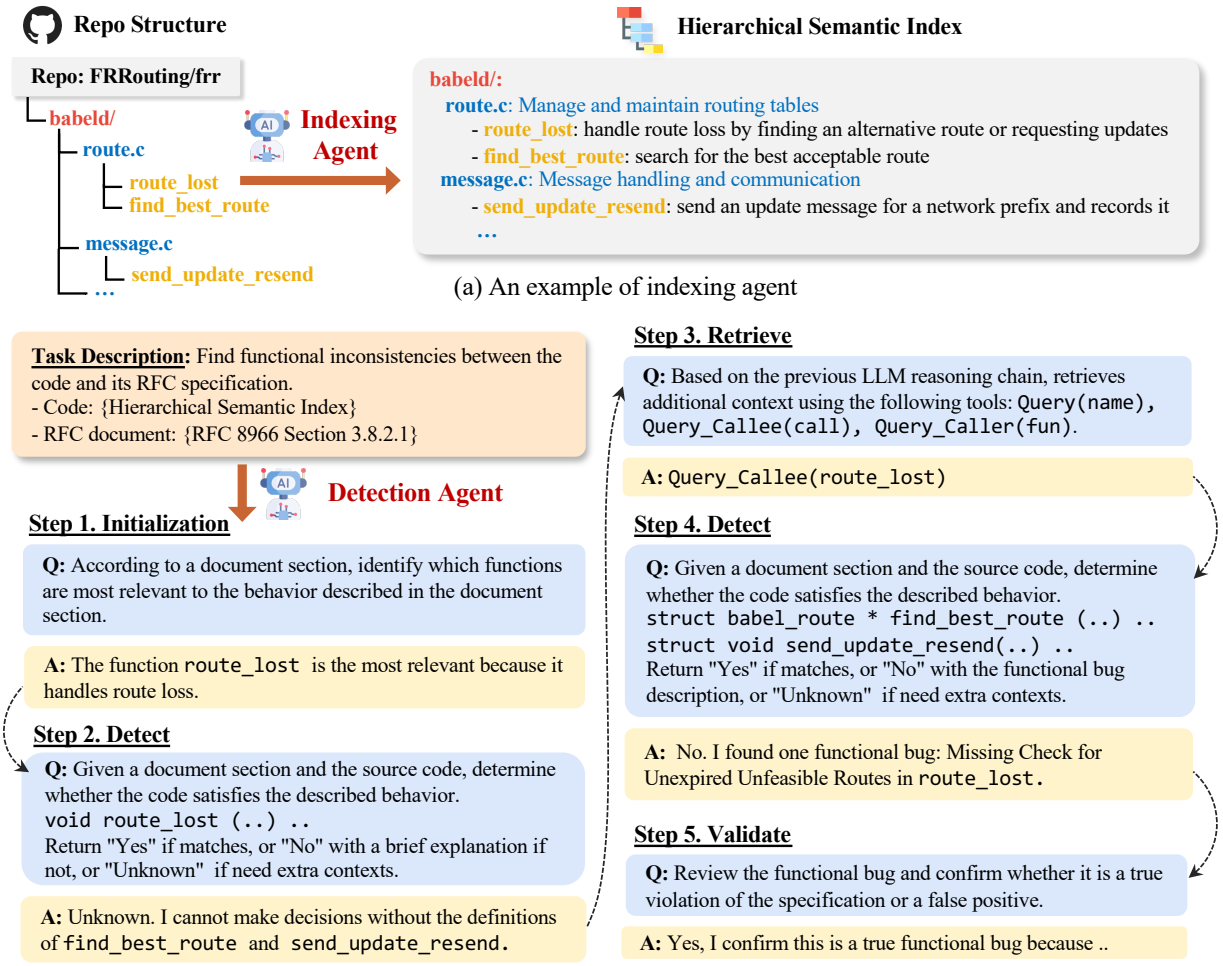
To bridge the gap, we propose RFCSCAN, an LLM-powered autonomous agent that identifies semantic inconsistencies between source code and RFC documents. Our approach is motivated by the observation that LLMs, having been pre-trained on vast corpora of natural language and programming code, exhibit strong capabilities in understanding both the specification offered by the RFC document and program semantics in the code. For instance, given the semantic property shown in Figure 1(a) and the buggy implementation in Figure 1(b), LLMs can potentially identify the inconsistency between the RFC documents and the corresponding implementation logic. However, applying LLMs for functional bug detection in network protocols is far from trivial, requiring us to resolve two critical technical challenges.

- Due to the lack of direct correspondence between RFC segments and source code, identifying where a high-level property is realized in the implementation is quite difficult. For instance, checking the property in Figure 1(a) requires finding the related function `route_lost` in Figure 1(b), which involves exploring a large codebase without clear guidance from the RFC document.
- Even after locating a relevant function, LLMs may still struggle to determine whether the implementation satisfies the intended property. Since a semantic requirement is often implemented through multiple interconnected functions spread across files or modules, LLMs typically need additional contexts, such as the functions `find_best_route` and `send_update_resend` in Fig. 1.

To tackle these challenges, we draw inspiration from how human developers typically audit functional correctness against protocol specifications. When verifying whether a particular semantic property is correctly implemented, experienced developers first perform a preliminary walkthrough of the codebase to become familiar with the roles of different functions, files, and modules. They then progressively gather additional program constructs, such as related functions or data structures, to reason about the correctness of the implementation and identify potential violations.

RFCSCAN mirrors this human auditing process through two key technical components: ***code semantic indexing*** and ***retrieval-guided detection***. The former creates a semantic map of the codebase by summarizing the functionality of functions, files, and directories in natural language, allowing fast and informed localization of potentially relevant code. The latter incrementally augments the reasoning context with related program constructs, enabling the system to validate a property when its implementation spans multiple functions or modules. These two techniques are realized through two collaborative agents in RFCSCAN, the ***indexing agent*** and the ***detection agent***, whose respective workflows are illustrated in Figure 2.

- ***Indexing Agent.*** The indexing agent performs semantic summarization and generates *hierarchical semantic indexes* at the directory, file, and function levels. As shown in Figure 3 (a), for the Babel protocol, files like `route.c` and `message.c`, as well as functions like `route_lost`, `find_best_route`, and `send_update_resend`, are annotated with concise summaries of their functionality. These semantic descriptions are later utilized by the detection agent to navigate and localize relevant code.
- ***Detection Agent.*** Given a semantic property, the detection agent uses hierarchical indexes to identify inconsistencies. As shown in Figure 3(b), it proceeds with four actions in five steps: (1) Use the semantic indexes to localize a relevant function, such as `route_lost`; (2) Detect inconsistencies based on this context; (3) Retrieve additional related functions, such as `find_best_route` and




(a) An example of indexing agent

(b) An example of detection agent

**Fig. 3: Illustrative examples of the two agents in** RFCSCAN

`send_update_resend`, if the context is insufficient; (4) Draw a conclusion about a potential violation; and (5) Apply a self-critique strategy to reassess the reasoning for hallucination mitigation.

## III. OUR APPROACH

RFCSCAN emulates the reasoning process of human developers when auditing for functional bugs and consists of two collaborative agents, the indexing agent and the detection agent, which support scalable and effective functional bug detection upon network protocol implementations. In what follows, we present more technical details of the two agents with concrete prompts and illustrative examples.

### A. Phase 1: Code Semantic Indexing

To bridge RFC documents with relevant code segments, we follow the behavior of human developers by constructing semantic indexes over the protocol implementation. This approach enables us to capture the semantics of the code in a manner analogous to how developers form a high-level understanding of the implementation. Basically, a simple indexing design is to utilize the names of functions, files, and directories as summaries, as they often suggest the intended functionality [22], [23]. However, such names are not informative enough to capture precise semantics. For example, in the Babel implementation shown in Figure 1, both `babeld.c` and `babel_main.c` reside in the same directory, yet their names do not clearly reveal their distinct functionalities. To generate more informative semantic indexes, we leverage LLMs to summarize the contents of functions, files, and directories, producing concise natural language descriptions that form a hierarchical semantic view of the implementation.

As shown in Figure 4, the indexing agent constructs hierarchical semantic indexes in a bottom-up manner. It begins at the *function level*, where each function definition is passed to the LLM with a prompt, which asks for a concise summary of its semantics. At the *file level*, summaries of all functions within the same source file are aggregated and used to prompt the LLM for a description of the overall functionality of the file, capturing how individual behaviors contribute to the role of the file in the protocol implementation. This process continues at the *directory level*, where the agent combines summaries of all files and subdirectories to generate a higher-level description of the directory. Finally, summaries from all directories are aggregated into a repository-level summary, which is a top-level directory summary, reflecting the system's

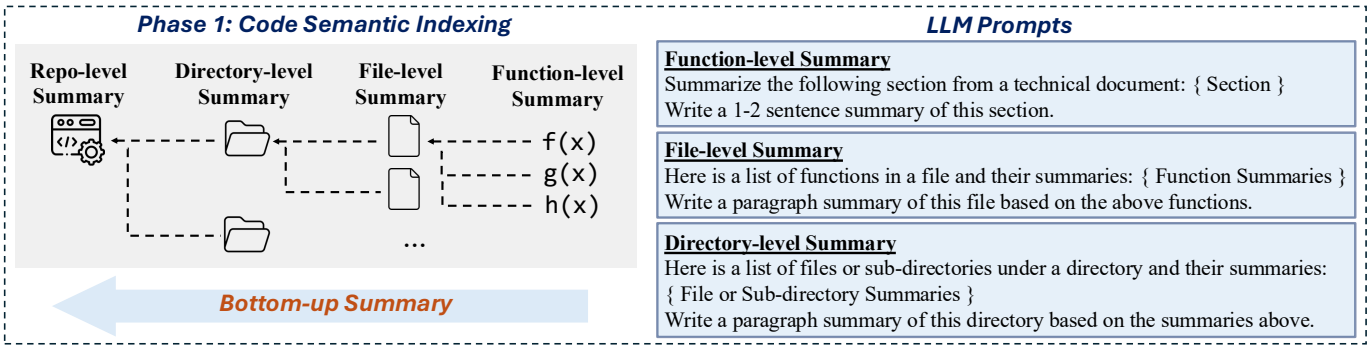



**Fig. 4: The workflow of code semantic indexing and prompt templates**

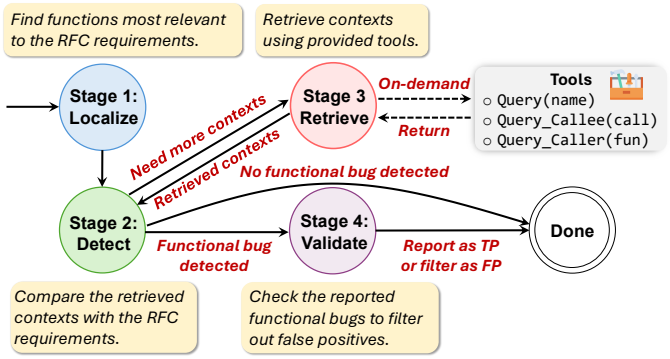



**Fig. 5: The state machine of the detection agent**

overall structure and responsibilities. This bottom-up indexing process produces a structured semantic index aligned with the code hierarchy, enabling precise and efficient retrieval during detection. The corresponding prompt templates for each indexing level are shown on the right side of Figure 4. Notably, the code semantic indexing is one-time effort. When the codebase changes, we can incrementally update the indexes. In our evaluation, we choose the Claude 3.5 Sonnet model and the average cost is 1.89$, shown in Table V. We can also consider using cheaper models for indexing as the task is relatively simple and does not rely on strong reasoning ability.

**Example 1.** Consider the repository structure shown on the left side of Figure 3(a). We first ask the LLMs to summarize each function. For example, `route_lost` handles route loss, `find_best_route` searches for the best acceptable route, and `send_update_resend` sends and records update messages. Next, the function-level summaries under the same file are combined to produce a file-level summary that captures the main responsibility of each file. In this case, we can discover that `route.c` focuses on route maintenance, and `message.c` on inter-node communication. Finally, all file summaries are merged into a directory-level summary for the `babeld` module. Lastly, we can obtain the hierarchical semantic indexes shown on the right side of Figure 3(b).

### *B. Phase 2: Retrieval-guided Detection*

As demonstrated in Section III-A, the hierarchical semantic indexes allow us to effectively bridge RFC documents with their corresponding implementation functions. Similar to how human developers audit code, the analysis often requires collecting additional program constructs, such as caller/callee functions and data structure definitions, to augment the context until a reliable conclusion can be drawn regarding the conformance or violation of a given semantic property. Building on this insight, we introduce the detection agent, which performs retrieval-guided detection to identify inconsistencies between the RFC specification and the code as functional bugs.

Technically, the retrieval-guided detection begins by preprocessing the RFC document and segmenting it into sections and subsections based on its structural headings. These segments are considered the semantic properties that guide the detection process. For each semantic property, the detection agent scans the implementation by following a workflow similar to a finite state machine, as illustrated in Figure 5. Specifically, it first localizes relevant functions for inspection (***Localization***), determines whether the implementation satisfies the RFC requirement (***Detection***), and, if necessary, retrieves additional context (***Retrieval***). When a potential functional bug is found, the detection agent examines the reasoning chain that produces the reported bug via self-criticism (***Validation***). In the following subsections, we provide further details on each component of the detection workflow shown in Figure 5.

***Localization.*** Guided by the pre-built hierarchical semantic indexes, the detection agent first identifies functions that are likely responsible for implementing the behavior described in a given RFC section. This localization is performed in a top-down manner. Starting from the root directory of the protocol implementation, it navigates through different levels of directories and files by instantiating the prompt template shown in Figure 6(a). At each level, it reviews the semantic summaries of directories or files, selects those most relevant to the RFC section, and descends into the selected entries. Upon reaching a file, it examines the summaries of its functions and identifies those that align with the described behavior. Because this search process is recursive, the final set of relevant functions may span multiple files across the codebase.

***Detection.*** Based on the localized relevant functions, the detection agent attempts to identify inconsistencies between the implementation and the RFC. Using the template in Figure 6(b), it queries the LLM to make one of three decisions, as shown in Figure 5. Specifically, if the LLM detects a

**Localization Prompt**
According to a document section, identify which entries are most relevant to the behavior described in the document section.
{ Document Section } { Entry Summaries }
Return a list of names: ["file.c", "subdir"] or ["fun1", "fun2"].

**(a) Prompt used for Localization**

**Detection Prompt**
Given a document section and the source code, determine whether the code satisfies the described behavior. { Document Section } { Code }
Return "Yes" if matches, or "No" with a brief explanation if not, or "Unknown " if need extra contexts.

**(b) Prompt used for Detection**

**Retrieval Prompt**
Based on the previous LLM reasoning chain, retrieves additional context using the following tools: `Query(name)`, `Query_Callee(call)`, `Query_Caller(fun)`.
{ LLM Reasoning Chain }

**(c) Prompt used for Retrieval**

**Validation Prompt**
Review the functional bug and confirm whether it is a true violation of the specification or a false positive.
{ Document Section } { Code } { Functional Bug }
If the functional bug is valid, confirm it. If not, explain why.

**(d) Prompt used for Validation**

**Fig. 6: The prompt templates used in the detection agent**

potential functional bug, the agent proceeds to validate it (see the *Validation* stage below). If the LLM concludes that the implementation conforms to the RFC requirement, the agent terminates the analysis for the current property. Otherwise, the current context is insufficient, the agent initiates an additional retrieval to expand the context (see the *Retrieval* stage below). Notably, the detection agent resembles the reasoning pattern of human developers, who iteratively gather evidence from the program constructs to identify potential bugs or to justify the correctness of an implementation.

***Retrieval.*** To assist the LLM in identifying functional bugs or justifying correctness, the detection agent enables demand-driven retrieval of additional program constructs. This is achieved by equipping the LLM with a set of predefined tools, which it can invoke through function calls as needed. Specifically, the agent provides three types of tools. First, `Query(name)` retrieves the definition of a specific data structure, macro, or global variable. This is useful when the current context references symbols whose definitions are not yet available. Second, `Query_Callee(call)` returns the definition of the callee function invoked at the specified call site. This is used when the current function doesn't provide enough information to determine whether the behavior matches the RFC, and the LLM suspects that the called function contains important logic. This selective strategy enables the LLM to focus on semantically meaningful functions while avoiding unnecessary expansion of trivial or unrelated calls. Third, `Query_Caller(fun)` retrieves all callers of the function `fun`, allowing the LLM to examine how the function is used, particularly whether its preconditions are satisfied by the callers. These retrieval operations are triggered on demand, guided by the LLM's reasoning. Benefiting from model's planning ability, the LLMs can choose the proper tools for retrieval. The retrieved results further augment the analysis context in the *Detection* stage for continued analysis. The LLM prompt is shown in Figure 6(c).

***Validation.*** Due to the potential lengthy detection context, the LLMs may hallucinate and report a false positive. To mitigate hallucination, we employ self-critics [24] in the detection agent. Specifically, the detection agent reviews all available information, including the RFC section, the retrieved context, and earlier reasoning steps, to make a final judgment. If the violation is confirmed, the functional bug is reported as a true positive. Otherwise, it is discarded as a false positive. It may also uncover bugs missed during previous detection. The prompt is instantiated using the template in Figure 6(d). Our results in Section IV-E show that the self-critics can reduce the false positives by 71.9% (from 64.3% to 18.1%).

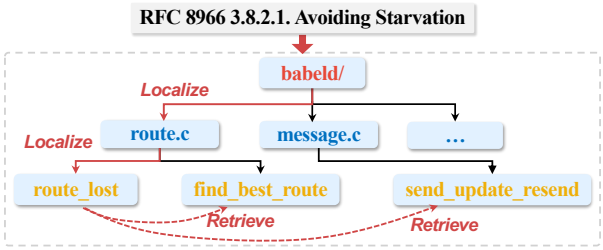



**Fig. 7: An illustrative example of Phase 2**

**Example 2.** Given the RFC section in Figure 1(a) and the semantic indexes in Figure 3(a), the detection agent identifies the relevant function `route_lost` in `route.c` (Figure 7). Finding its context insufficient, the agent retrieves `find_best_route` and `send_update_resend`, which select candidate routes and issue reactive updates, respectively. Notably, `send_update_resend` resides in a different file. With this broader context, the agent observes that only feasible routes are searched (`flag = 1`) and no sequence number request is made in the fallback logic, violating the starvation prevention rule. Retrieval stops here, and the bug is identified as shown in Figure 1 and detailed in Figure 3(b).

## IV. Implementation and Evaluation

RFCSCAN is implemented atop AutoGen [25], a multi-agent framework designed for constructing LLM applications. To extract functional specifications, we collect RFC documents in plain text format and derive informal functional properties through structured segmentation of the textual content. We employ Claude 3.5 Sonnet [26] as the underlying LLM for RFCSCAN, configured with a temperature of 0.0 to enforce greedy decoding. This setting reduces randomness in model responses, thereby enhancing the consistency and reproducibility of RFCSCAN. To facilitate the indexing and detection agents,

**TABLE I: The statistics of evaluation subjects.**

| Name | Description | LoC | Version (#Page) |
|---|---|---|---|
| Babel | Distance-vector Routing Protocol | 9.6K | RFC 8966 (54) |
| BFD | Bidirectional Forwarding Detection Protocol | 17.3K | RFC 5880 (49) |
| NHRP | NBMA Next Hop Resolution Protocol | 9.2K | RFC 2332 (52) |
| RIPng | Routing Information Protocol Next Generation | 9.3K | RFC 2080 (19) |
| DHCP | Dynamic Host Configuration Protocol | 8.6K | RFC 2131 (45) |
| IGMP | Internet Group Management Protocol | 5.2K | RFC 2236 (24) |

**TABLE II: The statistics of main results.**

| Protocols | #Incons. | TP | Precision | Unique Bug | | #Total Bug |
|---|---|---|---|---|---|---|
| | | | | #New | #Old | |
| Babel | 16 | 12 | 75.0% | 8 | 2 | 10 |
| BFD | 30 | 28 | 93.3% | 9 | 5 | 14 |
| NHRP | 12 | 10 | 83.3% | 10 | 0 | 10 |
| RIPng | 3 | 2 | 66.7% | 2 | 0 | 2 |
| DHCP | 12 | 8 | 66.7% | 4 | 0 | 4 |
| IGMP | 10 | 8 | 80.0% | 7 | 0 | 7 |
| Total | 83 | 68 | 81.9% | 40 | 7 | 47 |

we implement a suite of analysis tools using Tree-sitter [27], a parsing library for different programming languages. In particular, we construct call graphs by leveraging function names and the numbers of their associated parameters or arguments. The output generated by the indexing agent is stored in a JSON file, enabling incremental analysis as the protocol implementation evolves. This design avoids redundant processing by skipping the analysis of unchanged functions, files, and modules.

To assess the performance of RFCSCAN, we conduct experiments to address the following research questions:

- **RQ1**: How effective does RFCSCAN identify functional bugs in real-world network protocol implementations?
- **RQ2**: How does RFCSCAN compare to existing approaches?
- **RQ3**: What are the runtime and token costs of RFCSCAN?
- **RQ4**: How do the two agents contribute to performance?

### A. Dataset

We evaluate RFCSCAN on six protocol implementations from two widely-used, open-source network stacks: FRRouting [28] and lwIP [29]. FRRouting (FRR) is an Internet routing protocol suite for Linux and Unix platforms, with over 3.6K stars on GitHub. It is deployed in production by major infrastructure providers such as NVIDIA and Orange, and is distributed through mainstream Linux repositories including Debian and Fedora. lwIP is a lightweight TCP/IP stack designed for resource-constrained environments, with over 1.3K stars on GitHub. It is widely integrated into embedded systems, IoT platforms, and real-time applications, and has been adopted by vendors like Intel, Xilinx, and Freescale. Overall, both FRRouting and lwIP are actively maintained and widely used in practice, making them ideal subjects for assessing RFCSCAN across diverse deployment contexts. For each protocol, we collect the corresponding RFC that the implementation is based on as the reference. The protocol implementation sizes range from 5.2 KLoC to 17.3 KLoC, whereas the corresponding RFC documents span between 19 and 54 pages. The detailed information is shown in Table I.

### B. RQ1: Effectiveness of RFCSCAN

**Setup and Metrics.** To answer RQ1, we evaluate RFCSCAN upon each protocol implementation and the corresponding RFC. For each protocol, we collect the functional bugs reported by RFCSCAN and manually verify whether each one reflects a real violation. Then we report the numbers of true positives and false positives, and calculate the false positive rate. Since a single underlying issue may result in multiple reported bugs—e.g., when the same property is described across multiple sections of the RFC—we group related reports into unique bugs. For each unique bug, we check the project's issue tracker and pull requests to determine whether it is a previously unreported bug (i.e., new) or a known issue (i.e., old) that has not yet been fixed in the latest version.

**Results.** As shown in Table table II, RFCSCAN generates a total of 83 bug reports across six protocols, of which 67 are true positives, yielding an overall precision of 81.9%. The false positive rate remained low, with most protocols under 25%, and BFD exhibiting the lowest at only 10.0%. Grouping related bug reports, we uncovered 47 unique bugs, including 40 new bugs and 7 already reported ones. The complete bug list is shown in Table III. To date, 20 out of 47 unique bugs have been confirmed or their patches have been approved or merged by the developers. Overall, these results demonstrate that RFCSCAN is effective in identifying bugs across diverse protocols. To better understand the diversity of functional bugs detected by RFCSCAN, we categorize the 47 unique bugs and obtain seven categories, namely *Parsing*, *Config*, *Routing*, *Err Handle*, *State*, *Timer*, and *Demultiplex*. As shown in Table III, most bugs fall under State (15/47) and Parsing (11/47), highlighting common pitfalls in maintaining protocol state and validating input structure. However, RFCSCAN also discovered bugs such as Timer misconfiguration (e.g., incorrect IGMP delays in Bug #46), Demultiplexing issues (e.g., Incorrect BFD session look up in Bug #16), and Err Handle gaps (e.g., missing error-on-error prevention in Bug #32). This variety highlights the ability of RFCSCAN to detect diverse semantic violations specified by the RFCs.

### C. RQ2: Baseline Comparison

**Setup and Metrics.** To the best of our knowledge, no prior work targets functional bug detection in network protocols based on RFCs. We compare RFCSCAN with GitHub Copilot using three LLMs: Claude 3.7 Sonnet (Thinking Mode), Claude 3.5 Sonnet, and GPT-4o. All baselines are accessed via Copilot's Workspace Chat interface in Visual Studio Code, which supports repository-level queries. We prompt Copilot to identify deviations between protocol implementations and RFC documents. To ensure a fair comparison with RFCSCAN, which analyzes one RFC section at a time, we provide Copilot with one section at a time in a single prompt. For GitHub Copilot with Claude 3.7 Sonnet, we manually examine all generated bug reports. However, Claude 3.5 Sonnet and GPT-4o produce hundreds of candidates, making full manual verification impractical. To avoid over-reporting and minimize false positives, we instructed each LLM baseline to only

**TABLE III: The list of unique bugs.**

| ID | Protocol | Type | Description | New? | Status |
|---|---|---|---|---|---|
| 1 | BABEL | Routing | Request forwarding does not prioritize feasible routes. | ✓ | PR Merged |
| 2 | BABEL | Routing | Miss check unexpired unfeasible routes to avoid starvation. | ✓ | PR Merged |
| 3 | BABEL | Config | Miss source port validation before parsing. | ✓ | PR Merged |
| 4 | BABEL | Config | Missing router-id validation in babel_get_myid. | ✓ | PR Merged |
| 5 | BABEL | Parsing | known_ae() function accepts incorrect AE values. | ✓ | PR Merged |
| 6 | BABEL | Parsing | Incorrect parsing of type field in parse_request_subtlv. | ✓ | PR Merged |
| 7 | BABEL | Parsing | Handle unknown TLVs incorrectly in parse_request_subtlv. | ✓ | Pending |
| 8 | BABEL | Parsing | Miss mandatory bit handling in parse_ihu_subtlv. | ✓ | Pending |
| 9 | BABEL | Parsing | Miss Router-Id validation for RouterId TLV. | ✗ | PR Approved |
| 10 | BABEL | Parsing | Miss Router-Id Validation for Seqno Request TLV. | ✗ | PR Approved |
| 11 | BFD | Parsing | Miss check Simple Password Authentication format. | ✗ | Confirmed |
| 12 | BFD | Parsing | Miss check Keyed MD5 Authentication format. | ✗ | Confirmed |
| 13 | BFD | Parsing | Miss check Meticulous Keyed MD5 Authentication format. | ✗ | Confirmed |
| 14 | BFD | Parsing | Miss check Keyed SHA1 Authentication format. | ✗ | Confirmed |
| 15 | BFD | Parsing | Miss check Meticulous Keyed SHA1 Authentication format. | ✗ | Confirmed |
| 16 | BFD | Demultiplex | Incorrect session lookup for demultiplexing. | ✓ | PR Merged |
| 17 | BFD | State | Miss state validation for demand bit in ptm_bfd_send. | ✓ | Pending |
| 18 | BFD | Demultiplex | Missing poll Sequence for demand bit changes. | ✓ | Pending |
| 19 | BFD | Timer | Incorrect initialization of DesiredMinTxInterval. | ✓ | PR Approved |
| 20 | BFD | Config | Miss check passive mode with zero discriminator. | ✓ | PR Merged |
| 21 | BFD | State | LocalDiag not being set. | ✓ | PR Merged |
| 22 | BFD | State | Fail to maintain session state even when a session is down. | ✓ | Pending |
| 23 | BFD | State | Fail to start or stop Demand mode at any time. | ✓ | Pending |
| 24 | BFD | Timer | Miss check increased transmission interval. | ✓ | PR Merged |
| 25 | RIPng | Routing | Incorrect address selection for non-RIPng port requests. | ✓ | Pending |
| 26 | RIPng | Routing | Miss link-local route filtering before adding into RTE. | ✓ | Pending |
| 27 | NHRP | Routing | Miss hop count validation in nhrp_packet_pull(). | ✓ | PR Merged |
| 28 | NHRP | State | Non-volatile request ID storage violation. | ✓ | Pending |
| 29 | NHRP | State | No storage/checking of uniqueness flag. | ✓ | Pending |
| 30 | NHRP | State | No rejection of duplicate unique registrations. | ✓ | Pending |
| 31 | NHRP | State | Miss uniqueness conflict detection and error handling. | ✓ | Pending |
| 32 | NHRP | Err Handle | Fail to prevent sending error indication pkts to error pkts. | ✓ | Pending |
| 33 | NHRP | Err Handle | Fail to guarantee single error per packet rule. | ✓ | Pending |
| 34 | NHRP | State | Miss Purge Request handler. | ✓ | Pending |
| 35 | NHRP | Routing | Miss loop detection for Responder Address Extension. | ✓ | Confirmed |
| 36 | NHRP | Routing | Fails to drop packets with NHS's own source addresses. | ✓ | Pending |
| 37 | DHCP | State | Miss response handling for DHCPINFORM messages. | ✓ | Pending |
| 38 | DHCP | State | DHCPREQUEST does not reuse the secs field. | ✓ | Pending |
| 39 | DHCP | State | Incorrect hooking message type in REBINDING stage. | ✓ | Pending |
| 40 | DHCP | Timer | Violation of T1/T2 timer relationship requirement. | ✓ | Pending |
| 41 | IGMP | Timer | Incorrect timer range coverage due to randomization error. | ✓ | Pending |
| 42 | IGMP | State | Miss version state tracking and handling per interface. | ✓ | Pending |
| 43 | IGMP | Timer | Incorrect IGMPv1 Max Response Time handling. | ✓ | Pending |
| 44 | IGMP | State | Missing report version selection logic. | ✓ | Pending |
| 45 | IGMP | State | Incorrectly allow IGMP join from invalid states. | ✓ | Pending |
| 46 | IGMP | Timer | Use a 500ms delay instead of the RFC-mandated 10 seconds. | ✓ | Pending |
| 47 | IGMP | Timer | Miss Version 1 router present timeout. | ✓ | Pending |

report bugs with 100% confidence. To enable a fair and scalable evaluation, we adopt a sampling-based evaluation. For each protocol, we randomly sample up to twice the number of bug reports identified by RFCSCAN. For example, since RFCSCAN generates 16 bug reports for Babel, we sample 32. If the model produces fewer than that, we evaluate all of them. Each sampled case is then manually reviewed by a domain expert. We do not compare with existing network protocol bug detection tools, as they either cannot detect functional bugs (e.g., ChatAFL [9], NetLifter [17]), or target narrow scope like parsing (e.g., ParCleanse [21], ParVAL [30]).

**Results.** Table IV shows the performance of GitHub Copilot under three model configurations. – With Claude 3.7 Sonnet (no sampling), it generated 215 bug reports, of which 78 were true positives (36.3% precision), uncovering 37 unique bugs (29 new, 8 old). – Claude 3.5 Sonnet achieved 19.9% precision over 538 inconsistencies and found 26 bugs. – GPT-4o returned the most bug reports (1,315) but the lowest precision (11.4%) and the fewest bugs (16). Overall, all three baselines suffer from low precision and high validation overhead. In contrast, RFCSCAN achieves 81.9% precision and discovers 47 bugs in total (40 new) with far less manual effort (Table II). RFCSCAN consistently reports the lowest false-positive rates and the highest bug counts across all protocols. Notably, RFCSCAN 's use of Claude 3.5 (a non-reasoning model) still outperforms Claude 3.7 (a reasoning model).

### *D. RQ3: Efficiency of* RFCSCAN

**Setup and Metrics.** To quantify the efficiency of RFCSCAN, we measure input/output token usage, financial cost, and execution time for analyzing each repository. The results are reported separately for Phase 1 (Code Semantic Indexing) and Phase 2 (Retrieval-guided Detection). Although Claude 3.5 is used for both phases in our evaluation, Phase 1 primarily

**TABLE IV: Baseline comparison across three model configurations.**

| Protocols | Copilot + Claude 3.7 Sonnet | | | | | Copilot + Claude 3.5 Sonnet (With Sample) | | | | | Copilot + GPT-4o (With Sample) | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | #Incons. | #TP | Precision | #New | #Old | #Incons. | #Sample | Precision | #New | #Old | #Incons. | #Sample | Precision | #New | #Old |
| Babel | 76 | 16 | 21.1% | 7 | 3 | 173 | 32 | 12.5% | 3 | 1 | 493 | 32 | 3.1% | 1 | 0 |
| BFD | 45 | 37 | 82.2% | 5 | 5 | 115 | 60 | 23.3% | 7 | 4 | 244 | 60 | 20.0% | 6 | 3 |
| NHRP | 27 | 10 | 37.0% | 9 | 0 | 89 | 24 | 20.8% | 4 | 0 | 204 | 24 | 16.7% | 4 | 0 |
| RIPng | 6 | 1 | 16.7% | 1 | 0 | 45 | 6 | 16.7% | 1 | 0 | 84 | 6 | 0.0% | 0 | 0 |
| DHCP | 32 | 9 | 28.1% | 4 | 0 | 64 | 24 | 25.0% | 4 | 0 | 188 | 24 | 42.0% | 1 | 0 |
| IGMP | 29 | 5 | 17.2% | 3 | 0 | 52 | 20 | 15.0% | 2 | 0 | 102 | 20 | 5.0% | 1 | 0 |
| **Total** | 215 | 78 | **36.3%** | 29 | 8 | 538 | 166 | **19.9%** | 21 | 5 | 1315 | 166 | **11.4%** | 13 | 3 |

**TABLE V: Token usage (In: input tokens, Out: output tokens), financial cost, and execution time per protocol.**

| Protocol | Phase 1: Code Semantic Indexing | | | Phase 2: Retrieval-guided Detection | | | Total | | |
|---|---|---|---|---|---|---|---|---|---|
| | Tokens (In/Out) | Cost ($) | Time (min) | Tokens (In/Out) | Cost ($) | Time (min) | Tokens (In/Out) | Cost ($) | Time (min) |
| Babel | 261K / 71K | 1.85 | 44 | 1340K / 41K | 4.63 | 39 | 1061K / 112K | 6.48 | 83 |
| BFD | 449K / 119K | 3.13 | 71 | 1086K / 30K | 3.71 | 52 | 1535K / 149K | 6.84 | 123 |
| NHRP | 319K / 81K | 2.17 | 49 | 960K / 25K | 3.26 | 56 | 1279K / 106K | 5.43 | 105 |
| RIPng | 197K / 52K | 1.37 | 31 | 429K / 9K | 1.42 | 23 | 626K / 61K | 2.79 | 54 |
| DHCP | 248K /44K | 1.40 | 28 | 1353K / 24K | 4.42 | 47 | 1601K / 68K | 5.82 | 75 |
| IGMP | 248K / 28K | 1.40 | 28 | 390K / 16K | 1.41 | 20 | 638K / 44K | 2.81 | 48 |
| **Average** | 287K / 66K | 1.89 | 42 | 926K / 24K | 3.14 | 39 | 1123K / 90K | 5.03 | 81 |

**TABLE VI: Ablation Studies.**

| Protocol | Ablation Study 1 | | | Ablation Study 2 | | | Ablation Study 3 | | |
|---|---|---|---|---|---|---|---|---|---|
| | Precision | #New | #Old | Precision | #New | #Old | Precision | #New | #Old |
| Babel | 52.6% | 5 | 3 | 40.0% | 8 | 2 | 33.3% | 10 | 3 |
| BFD | 68.8% | 8 | 5 | 70.6% | 5 | 5 | 67.9% | 10 | 5 |
| NHRP | 20.0% | 2 | 0 | 45.0% | 6 | 2 | 20.4% | 7 | 0 |
| RIPng | N/A | 0 | 0 | 25.0% | 1 | 0 | 5.9% | 1 | 0 |
| DHCP | 0% | 0 | 0 | 50.0% | 4 | 0 | 30.0% | 4 | 0 |
| IGMP | 57.1% | 3 | 0 | 18.2% | 2 | 0 | 29.6% | 4 | 0 |
| **Total** | **51.4%** | 18 | 8 | **44.7%** | 26 | 9 | **35.7%** | 36 | 8 |

involves lightweight summarization. Since it does not require complex reasoning, it could be replaced by smaller or less expensive LLMs with minimal impact on performance.

**Results.** As shown in Table V, RFCSCAN uses an average of 1123K input and 90K output tokens, costs $5.03, and completes each protocol analysis in 81 minutes. Since the analysis has a complete summarization of the whole code base and compares code against the full RFC, input token usage and cost largely depend on the code base size and RFC length. Overall, RFCSCAN is efficient for functional bug detection.

### *E. RQ4: Ablation Studies*

**Ablation Study 1: Without Code Semantic Indexing.** To evaluate the impact of code semantic indexing, we disable semantic summarization in Phase 1. The agent still follows the directory structure and explores the codebase hierarchically but no longer uses LLM-generated summaries. Instead, it relies solely on directory and file names plus function signatures. This setup follows the same design of AGENTLESS [23] and LocAgent [22], which uses code structures without semantic summaries. As shown in Table VI, removing semantic indexing significantly reduces precision (81.9% to 51.4%) and results in fewer detected bugs (47 to 26). This highlights the importance of semantic summaries in guiding the agent to more relevant code regions and reducing false positives.

**Ablation Study 2: Detection Without Retrieval.** To evaluate the impact of the *Retrieval* stage, we disable all code retrieval tools (`Query`, `Query_Callee`, `Query_Caller`). The detection agent must make decisions using only the initially retrieved function(s), without fetching additional context. As shown in Table VI, this results in a sharp drop in precision (from 81.9% to 44.7%) and fewer detected bugs (from 47 to 35). This shows that retrieval is important for reducing false positives and improving detection quality.

```
struct bfd_session *bfd_find_disc(struct sockaddr_any *sa,
                                  uint32_t ldisc) {
-  struct bfd_session *bs = bfd_id_lookup(ldisc);
-  switch (bs->key.family) {
-     case AF_INET:
-          if (memcmp(&sa->sa_sin.sin_addr, ..)) return NULL;
-     ..
-  }
-  return bs;
+  return bfd_id_lookup(ldisc);
 }
```

(a) Bug #16: Incorrect session look up

```
void bfd_set_shutdown(struct bfd_session *bs, bool shutdown) {
   if (shutdown) {
       SET_FLAG(bs->flags, BFD_SESS_FLAG_SHUTDOWN);
+      bs->local_diag = BD_ADMIN_DOWN; ..
   }
}
```

(b) Bug #21: LocalDiag not being set

**Fig. 8: Case Studies.**

**Ablation Study 3: Detection Without Self-critics.** To evaluate the impact of self-critics, we disable it during detection. This leads to a higher false positive rate (18.1% to 64.3%) and fewer bugs (47 to 44), emphasizing its crucial role in enhancing detection precision and finding overlooked bugs.

### *F. Case Studies*

**Bug #16: Incorrect session look up** (*Demultiplex*). As shown in Figure 8(a), the original `bfd_find_disc` performed an additional check on the peer's source address after looking up a session by discriminator. This directly

contradicts RFC 5880, which mandates that BFD sessions must be demultiplexed only using the *Your Discriminator* field. Address-based filtering may reject valid sessions when a peer's source address changes (e.g., due to routing updates or NAT), leading to unintended session loss and network disruption. We submitted a patch to remove the non-compliant address check, and it has been merged by the developers.

**Bug #21: LocalDiag not being set** (*State*). In Figure 8(b), the original `bfd_set_shutdown` function updated the SessionState but failed to assign `local_diag`. This violated RFC 5880, which requires setting `LocalDiag` to `ADMIN_DOWN` when transitioning to the `AdminDown` state. This may cause peers to misinterpret administrative shutdowns as faults, triggering unnecessary recovery actions and destabilizing the network. We submitted a patch to set `local_diag`, which has been merged by the developers.

### G. Threats to Validity.

Several threats may affect the validity of our findings. First, we adopt Claude 3.5 Sonnet as our language model with a temperature setting of 0.0 for stable analysis results, thereby enhancing the reproducibility of RFCSCAN. Nevertheless, alternative models or configurations may yield variations in performance. So we ran controlled experiments using two additional general-purpose API models: GPT-4o and DeepSeek-V3, to analyze the BFD protocol in FRRouting, where RFCSCAN discovered the most bugs. RFCSCAN detected 11 and 13 true bugs with the precision of 71.4% and 70.3%, respectively, demonstrating the effectiveness across models. Second, our manual classification of true positives and false positives may introduce subjective bias. To mitigate this risk, two authors independently reviewed each reported functional bug and resolved any disagreements through discussion. We further validated our results by reporting newly discovered bugs to protocol developers, where all bug reports that received a response have been confirmed as true positives. Third, our method relies on high-quality and well-structured documentation such as RFCs. Ambiguous or poorly maintained documentation might hinder semantic understanding, potentially threatening the effectiveness of our technique.

## V. Related Work

### A. Bug Detection in Network Protocol Implementation

The correctness of network protocol implementations is critical for ensuring secure and reliable digital communication. Existing bug detection techniques for protocol implementations can be broadly categorized into three approaches. First, fuzzing-based techniques—such as BooFuzz [31] and SAGE [32]—primarily identify bugs by triggering crashes during execution. Netlifter [17] and ChatAFL [9] improve fuzzing coverage by using static analysis or LLMs to infer packet formats and generate valid inputs, but still rely on crashes, overlooking subtle semantic issues. LLMIF [33] and mGPTFuzz [10] extract formats or FSMs from specifications using LLMs to uncover semantic bugs, but focus narrowly on state transition or input validation violations and are limited to specific IoT devices, hard to generalize to broader protocol testing. Second, differential analysis techniques [11]–[13] detect bugs by comparing the behaviors of multiple independent implementations of the same protocol. These approaches can uncover semantic discrepancies that fuzzing may miss. However, they are inherently limited to scenarios where multiple protocol implementations are available for comparison. Third, formal verification-based techniques rigorously check protocol implementations against formally specified properties. While powerful in reasoning about semantic correctness, they require substantial manual effort to construct, maintain, and validate formal models, posing significant barriers to practical deployment. In contrast to these prior efforts, our proposed technique, RFCSCAN, addresses functional correctness by bridging the informal specifications in RFC documents with the program semantics of the protocol implementation, facilitating the functional bug detection with high precision and efficiency.

### B. LLM-aided Static Bug Detection

LLM-aided static bug detection has advanced rapidly [34]–[40], typically following two directions. One uses LLMs to supply domain knowledge like function specifications [34]–[36] and bug definitions [37], to assist symbolic approaches in analyzing the program for bug detection. For example, IRIS leverages LLM-inferred sources and sinks to detect taint-style vulnerabilities [36], while KNighter synthesizes static analyzers from bug patches for kernel bug detection [37]. The other direction adopts agent-centric solutions that reason over source code directly without compilation [38]–[40]. Techniques such as LLMSAN [38] and LLMDFA [39] combine LLMs with SMT solvers and parsing-based analyzers, while LLift uses progressive prompting to find kernel vulnerabilities [40]. More recently, RepoAudit extends these efforts to detect multiple types of data-flow bugs [41]. In contrast, our work targets functional bugs, where bug specifications often lack clear formalization. The diverse nature of functional properties also prevents the application of standard static analysis frameworks. Nonetheless, our retrieval-based approach draws inspiration from traditional bug detection methodologies, enabling localizing code segments relevant to targeted functional properties, facilitating effective subsequent detection.

## VI. Conclusion

This paper presents RFCSCAN, an LLM agent designed for detecting functional bugs in network protocol implementations by identifying semantic inconsistencies between the code and RFC documents. Specifically, RFCSCAN comprises two complementary agents: an indexing agent responsible for semantic indexing of source code, and a detection agent performing retrieval-guided inconsistency detection. In our evaluation, RFCSCAN successfully uncovered 47 functional bugs across six different protocols, of which 20 have been acknowledged or fixed by the original developers. As the first LLM-based approach explicitly targeting functional bug detection in network protocols, our work offers valuable insights into future research on functional bug detection in domain-specific

software systems, demonstrating the significant potential of leveraging LLM capabilities in security auditing tasks.

## REFERENCES


[1] MITRE, "Cve-2020-1472," https://nvd.nist.gov/vuln/detail/CVE-2020-1472, 2020.

[2] Microsoft, "Netlogon remote protocol," https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-nrpc/ff8f970f-3e37-40f7-bd4b-af7336e4792f, 2024.

[3] A. A. de Amorim, C. Hritcu, and B. C. Pierce, "The meaning of memory safety," *CoRR*, vol. abs/1705.07354, 2017. [Online]. Available: http://arxiv.org/abs/1705.07354

[4] C. Cadar, D. Dunbar, and D. R. Engler, "Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation*, ser. OSDI '08. USENIX, 2008, pp. 209–224. [Online]. Available: https://www.usenix.org/conference/osdi-08/klee-unassisted-and-automatic-generation-high-coverage-tests-complex-systems

[5] S. Poeplau and A. Francillon, "Symbolic execution with SymCC: Don't interpret, compile!" in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 181–198. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/poeplau

[6] I. Yun, S. Lee, M. Xu, Y. Jang, and T. Kim, "QSYM : A practical concolic execution engine tailored for hybrid fuzzing," in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, pp. 745–761. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/yun

[7] P. Yao, J. Zhou, X. Xiao, Q. Shi, R. Wu, and C. Zhang, "Falcon: A fused approach to path-sensitive sparse data dependence analysis," *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, Jun. 2024. [Online]. Available: https://doi.org/10.1145/3656400

[8] H. Yan, Y. Sui, S. Chen, and J. Xue, "Spatio-temporal context reduction: a pointer-analysis-based static approach for detecting use-after-free vulnerabilities," in *Proceedings of the 40th International Conference on Software Engineering*, ser. ICSE '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 327–337. [Online]. Available: https://doi.org/10.1145/3180155.3180178

[9] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, ser. NDSS '24. The Internet Society, 2024.

[10] X. Ma, L. Luo, and Q. Zeng, "From one thousand pages of specification to unveiling hidden bugs: Large language model assisted fuzzing of matter iot devices," in *USENIX Security Symposium*. USENIX Association, 2024.

[11] M. Zheng, Q. Shi, X. Liu, X. Xu, L. Yu, C. Liu, G. Wei, and X. Zhang, "Pardiff: Practical static differential analysis of network protocol parsers," in *Proc. ACM Program. Lang.*, ser. OOPSLA '24. ACM, 2024, pp. 1208–1234.

[12] R. Rutledge and A. Orso, "Automating differential testing with over-approximate symbolic execution," in *2022 15th IEEE Conference on Software Testing, Verification and Validation*, ser. ICST '22. IEEE, 2022, pp. 256–266.

[13] G. S. Reen and C. Rossow, "Dpifuzz: a differential fuzzing framework to detect dpi elusion strategies for quic," in *Proceedings of the 36th Annual Computer Security Applications Conference*, ser. ACSAC '20. ACM, 2020, pp. 332–344.

[14] O. Udrea and C. Lumezanu, "Rule-based static analysis of network protocol implementations," in *USENIX Security Symposium*. USENIX Association, 2006.

[15] M. Musuvathi and D. R. Engler, "Model checking large network protocol implementations," in *NSDI*. USENIX, 2004, pp. 155–168.

[16] V.-T. Pham, M. Böhme, and A. Roychoudhury, "Aflnet: a greybox fuzzer for network protocols," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 2020, pp. 460–465.

[17] Q. Shi, J. Shao, Y. Ye, M. Zheng, and X. Zhang, "Lifting network protocol implementation to precise format specification with security applications," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. ACM, 2023, p. 1287–1301.

[18] Q. Shi, X. Xiao, R. Wu, J. Zhou, G. Fan, and C. Zhang, "Pinpoint: Fast and precise sparse value flow analysis for million lines of code," in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '18. ACM, 2018, pp. 693–706.

[19] Y. Sui and J. Xue, "SVF: interprocedural static value-flow analysis in LLVM," in *Proceedings of the 25th International Conference on Compiler Construction, CC 2016, Barcelona, Spain, March 12-18, 2016*, A. Zaks and M. V. Hermenegildo, Eds. ACM, 2016, pp. 265–266.

[20] G. Díaz, F. Cuartero, V. V. Ruiz, and F. L. Pelayo, "Automatic verification of the TLS handshake protocol," in *SAC*. ACM, 2004, pp. 789–794.

[21] M. Zheng, D. Xie, Q. Shi, C. Wang, and X. Zhang, "Validating network protocol parsers with traceable rfc document interpretation," in *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2025.

[22] Z. Chen, X. Tang, G. Deng, F. Wu, J. Wu, Z. Jiang, V. Prasanna, A. Cohan, and X. Wang, "Locagent: Graph-guided llm agents for code localization," *arXiv preprint arXiv:2503.09089*, 2025.

[23] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying llm-based software engineering agents," *arXiv preprint arXiv:2407.01489*, 2024.

[24] Z. Lin, Z. Gou, T. Liang, R. Luo, H. Liu, and Y. Yang, "CriticBench: Benchmarking LLMs for critique-correct reasoning," in *Findings of the Association for Computational Linguistics: ACL 2024*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 1552–1587. [Online]. Available: https://aclanthology.org/2024.findings-acl.91/

[25] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen LLM applications via multi-agent conversation," in *ICLR 2024 Workshop on Large Language Model (LLM) Agents*, 2024. [Online]. Available: https://openreview.net/forum?id=uAjxFFing2

[26] Anthropic, "Claude 3.5 sonnet," https://www.anthropic.com/claude/sonnet, 2025.

[27] "Tree-sitter," https://tree-sitter.github.io/tree-sitter/.

[28] F. community, "The frrouting protocol suite," https://github.com/FRRouting/frr, 2024.

[29] "lwip - a lightweight tcp/ip stack," https://github.com/lwip-tcpip/lwip, 2025.

[30] M. Zheng, D. Xie, and X. Zhang, "Large language models for validating network protocol parsers," *arXiv preprint arXiv:2504.13515*, 2025.

[31] J. Pereyda, "Boofuzz," https://github.com/jtpereyda/boofuzz, 2023.

[32] P. Godefroid, M. Y. Levin, and D. Molnar, "Sage: whitebox fuzzing for security testing," *Communications of the ACM*, vol. 55, no. 3, pp. 40–44, 2012.

[33] J. Wang, L. Yu, and X. Luo, "Llmif: Augmented large language model for fuzzing iot devices," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 881–896.

[34] C. Wang, J. Zhang, R. Wu, and C. Zhang, "Dainfer: Inferring API aliasing specifications from library documentation via neurosymbolic optimization," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 2469–2492, 2024. [Online]. Available: https://doi.org/10.1145/3660816

[35] C. Ye, Y. Cai, and C. Zhang, "When threads meet interrupts: Effective static detection of {Interrupt-Based} deadlocks in linux," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 6167–6184.

[36] Z. Li, S. Dutta, and M. Naik, "Llm-assisted static analysis for detecting security vulnerabilities," *CoRR*, vol. abs/2405.17238, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2405.17238

[37] C. Yang, Z. Zhao, Z. Xie, H. Li, and L. Zhang, "Knighter: Transforming static analysis with llm-synthesized checkers," *arXiv preprint arXiv:2503.09002*, 2025.

[38] C. Wang, W. Zhang, Z. Su, X. Xu, and X. Zhang, "Sanitizing large language models in bug detection with data-flow," in *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, Y. Al-Onaizan, M. Bansal, and Y. Chen, Eds. Association for Computational Linguistics, 2024, pp. 3790–3805. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.217

[39] C. Wang, W. Zhang, Z. Su, X. Xu, X. Xie, and X. Zhang, "LLMDFA: analyzing dataflow in code with large language models," in *Advances in Neural Information Processing Systems 38: Annual Conference on*

*Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds., 2024. [Online]. Available: http://papers.nips.cc/paper_files/paper/2024/hash/ed9dcde1eb9c597f68c1d375bbecf3fc-Abstract-Conference.html

[40] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "Enhancing static analysis for practical bug detection: An llm-integrated approach," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA1, Apr. 2024. [Online]. Available: https://doi.org/10.1145/3649828

[41] J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang, "Repoaudit: An autonomous llm-agent for repository-level code auditing," *arXiv preprint arXiv:2501.18160*, 2025.